# Operating Systems

Basics se CPU Scheduling tak | Hinglish study notes

**Is guide mein OS, system calls, processes, context switching aur CPU scheduling ko simple Roman Hinglish mein samjhaya gaya hai.**

## 1. Operating System kya karta hai?

Operating System (OS) user ke programs aur computer hardware ke beech interface hai. App file kholne, memory lene ya printer use karne ki request OS ko deti hai; OS us request ko hardware tak pahunchata hai aur result wapas deta hai.

- Convenience: computer ko use karna aasaan banana.
- Efficiency: CPU, memory, disk aur I/O devices ka behtar istemal karna.

Boot ke waqt BIOS/firmware OS ko load karne mein madad karta hai. OS ko user ke liye interface, hardware ke liye resource manager, aur system mein chalne wale program ke roop mein dekha ja sakta hai.

## 2. OS ke types aur kernel

Batch: milte-julte jobs ko group karke bina user interaction chalaya jaata hai. Multiprogramming: kai jobs memory mein hoti hain; ek I/O ka wait kare toh CPU doosri job chala sakta hai. Time-sharing: CPU ka time chhote turns mein users ko diya jaata hai. Real-time: system ko deadline ke andar response dena hota hai.

Network systems files/devices share karte hain; parallel systems mein ek machine par kai CPUs kaam karte hain; distributed systems mein alag machines milkar service deti hain.

Kernel OS ka central hissa hai. Yeh hardware-software communication aur resources manage karta hai. Kernel designs mein monolithic, microkernel, hybrid aur exokernel shamil hain.

## 3. System calls: user mode aur kernel mode

Applications aam taur par user mode mein chalti hain: unhe hardware ya doosre process ki memory par seedha unrestricted access nahi hota. OS ka privileged code kernel mode mein chalta hai.

Jab app ko file read karni ho, process banana ho ya device use karna ho, woh system call karti hai. Jaise read().

System call ka flow: app request deti hai; library syscall number/parameters set karke TRAP chalati hai; CPU kernel mode mein jaata hai; kernel request handle karke result deta hai; CPU user mode mein lautkar app ko resume karta hai. Mode boundary protection ke liye hai.

System calls process control, file/device management, time aur process information, communication, aur permissions jaise kaam karte hain.

## 4. Program, process aur PCB

Program disk par rakhi instructions hain. Process un instructions ka execution mein chal raha instance hai. Ek program ke kai processes ho sakte hain.

PCB (Process Control Block) har process ka record hai. Ismein process ID, state, priority, next instruction ka address, saved CPU registers, I/O details aur accounting information hoti hai. Process rukne par CPU state PCB mein save ho sakti hai.

## 5. Process states aur schedulers

New: process banne ki taiyari mein. Ready: memory mein hai aur CPU ka wait kar raha hai. Running: CPU par execute ho raha hai. Blocked/Waiting: I/O ya kisi event ka wait kar raha hai. Terminated: execution complete.

Bade state model mein Suspended Ready aur Suspended Blocked bhi hote hain: process temporarily memory se disk par swap hua hota hai.

Long-term scheduler jobs ko system/memory mein admit karta hai. Short-term scheduler ready queue se agla CPU process chunta hai. Medium-term scheduler (swapper) processes ko memory aur disk ke beech move karta hai. Dispatcher selected process ko CPU deta, context restore karta aur execution sahi jagah se resume karata hai.

## 6. Context switch

CPU ek process ko rok kar doosre ko chalata hai toh OS current process ka state save karta hai aur agle process ka state restore karta hai. Isse context switch kehte hain. Is dauraan application ka useful kaam nahi hota, isliye yeh overhead hai.

Mode switch sirf user/kernel privilege badalta hai. Context switch mein CPU par chalne wala process bhi badalta hai. Same process ke threads ke beech switch aksar process switch se tez hota hai, kyunki threads address space share karte hain.

## 7. CPU scheduling: terms aur formulas

AT (Arrival Time): process ready queue mein kab aaya. BT (Burst Time): process ko kitna CPU time chahiye. CT (Completion Time): process kab finish hua.

> **TAT (Turnaround Time) = CT - AT | WT (Waiting Time) = TAT - BT | RT (Response Time) = pehli baar CPU start - AT**

Preemptive scheduling mein OS running process se CPU wapas le sakta hai. Non-preemptive mein process CPU rakhta hai jab tak finish ya I/O ke liye block na ho.

## 8. FCFS, SJF aur SRTF

FCFS (First-Come, First-Served): jo pehle aaya woh pehle chalega. Simple hai, starvation nahi hoti. Lekin lamba process chhote processes ko rok sakta hai; ise convoy effect kehte hain.

SJF (Shortest Job First): CPU free hone par arrived processes mein sabse chhota burst choose hota hai. Yeh non-preemptive hai. Chhote jobs ki waiting kam hoti hai, lekin lambe jobs ko starvation ho sakti hai.

SRTF (Shortest Remaining Time First): SJF ka preemptive version. Naya process aaye toh uske burst ko running process ke remaining time se compare karte hain; chhota remaining time wala CPU leta hai. Waiting time kam ho sakta hai, lekin context switches zyada hote hain.

> **Deck ke same example data par average waiting time: FCFS = 4.75, SJF = 4, SRTF = 3. Yeh us input ka result hai; har workload mein yahi ranking zaroori nahi.**

## Ek line mein poora flow

OS resources sambhalta hai; process CPU maangta hai; scheduler process chunta hai; dispatcher CPU deta hai; zaroorat padne par context switch hota hai. Scheduling ka goal waiting/response kam rakhna, fairness maintain karna aur overhead ko sambhalna hai.